GitHub-репозиторий ВКР по веб-разработке

Репозиторий — часть результата ВКР

Демонстрация на ноутбуке автора заканчивается вместе с защитой

Репозиторий сохраняет код, конфигурацию и способ проверки

Новый пользователь должен получить результат без устных пояснений

Типичная ситуация после защиты

Внутри: client_final, server2 и database.sql

README: «Установите зависимости и запустите сервер»

Неясно, что разработано и как это проверить

Один проект читают разные люди

1. Комиссия сопоставляет заявленное решение с артефактами
2. Руководитель проверяет способ получения результата
3. Разработчик разворачивает систему на другом компьютере
4. Автор позднее восстанавливает версии и команды

Проверяемый путь

- Понять назначение
- Подготовить конфигурацию
- Поднять сервисы и применить миграции
- Запустить пользовательский сценарий
- Проверить результат и ограничения

Четыре вопроса за первые 60 секунд

- Цель: какую задачу решает система
- Состав: frontend, backend, база и внешние сервисы
- Запуск: требования, конфигурация и команды
- Проверка: URL, учетная запись и ожидаемый результат

Первые строки README задают контекст

Плохо: «Web project for diploma»

Хорошо: назвать задачу, пользователей и вклад автора

Указать состав системы и основной пользовательский сценарий

Сразу дать ссылку на стенд или отметить его состояние

Источник: GitHub Docs — About READMEs

Восемь критериев веб-репозитория

1. Назначение и результат
2. Стек и требования
3. Копируемый Quick Start
4. Конфигурация и секреты
5. Структура компонентов
6. База данных и миграции
7. API, тесты и CI
8. Деплой, безопасность и лицензия

1. Назначение и результат

- ✓ Какую проблему решает система
- ✓ Кто выполняет основной сценарий
- ✓ Что именно разработано автором ВКР
- ✓ Где находится демонстрация
- ✓ Что считать успешным результатом

2. Стек и требования

- ✓ Версии Node.js, backend runtime и СУБД
- ✓ Выбранный пакетный менеджер и lock-файлы
- ✓ Docker, Make или другие обязательные инструменты
- ✓ Поддерживаемые ОС и известные ограничения
- ✓ Не список технологий, а условия запуска

3. Quick Start без догадок

- ✓ git clone и переход в каталог
- ✓ копирование .env.example
- ✓ установка или сборка зависимостей
- ✓ запуск сервисов
- ✓ миграции и тестовые данные
- ✓ URL и команда остановки

4. Конфигурация и секреты

- ✓ .env.example перечисляет необходимые переменные
- ✓ .env исключен из Git
- ✓ **Production-пароли и токены не публикуются**
- ✓ Скомпрометированный ключ отзывают
- ✓ Безопасные значения позволяют выполнить учебный запуск

Источник: GitHub Docs — Best practices

5. Структура объясняет состав системы

frontend/ — клиентское приложение

backend/ — API и бизнес-логика

infra/ — конфигурация развертывания

docs/ — архитектура и решения

tests/ — интеграционные и E2E-проверки

compose.yml — локальные сервисы

6. База должна создаваться воспроизводимо

- ✓ Схема создается миграциями
- ✓ Seed формирует безопасные тестовые данные
- ✓ Указана команда сброса состояния
- ✓ Не публикуются реальные дампы и персональные данные
- ✓ Тестовая учетная запись позволяет повторить сценарий

7. Контракт и тесты подтверждают работу

- ✓ OpenAPI или документация фиксируют контракт
- ✓ Команда тестов указана в README
- ✓ Основной сценарий проверяется автоматически
- ✓ CI выполняет ключевые проверки
- ✓ Проверка соответствует заявленному результату

8. Публикация требует границ доверия

- ✓ Локальная и production-конфигурации разделены
- ✓ LICENSE определяет условия использования
- ✓ .gitignore и .dockerignore исключают лишнее
- ✓ Известные ограничения названы честно
- ✓ Указан способ сообщить об ошибке

Источник: GitHub Docs — Licensing a repository

Минимальная структура README

- Задача, пользователи и вклад автора
- Демонстрация и архитектура
- Требования и .env.example
- Quick Start
- Миграции, seed и тестовая учетная запись
- Тесты, деплой, ограничения и лицензия

Учебный антипример

- README из двух общих строк
- client_final и server2 без объяснений
- .env и production-ключ находятся в Git
- database.sql заменяет миграции
- Нет URL, тестовой учетной записи и тестов

Исправление начинается с блокеров

1. Отозвать секреты и удалить их из публикации
2. Добавить безопасную конфигурацию и запуск
3. Сделать схему и данные воспроизводимыми
4. Описать цель, URL и критерий успеха
5. Подтвердить основной сценарий тестом

Публичные примеры показывают разные приемы

GitHub Docs — требования и безопасность

Full Stack FastAPI Template — структура и развертывание

RealWorld — API-контракт и совместимость

beautify-md fixture — намеренно плохой README

Перед отправкой ссылки комиссии

- Команды проверены в чистом окружении
- Секретов и персональных данных нет
- Миграции и seed воспроизводят состояние
- URL и сценарий проверки указаны
- Основной путь подтвержден тестом или CI
- Хороший репозиторий — результат и маршрут к нему